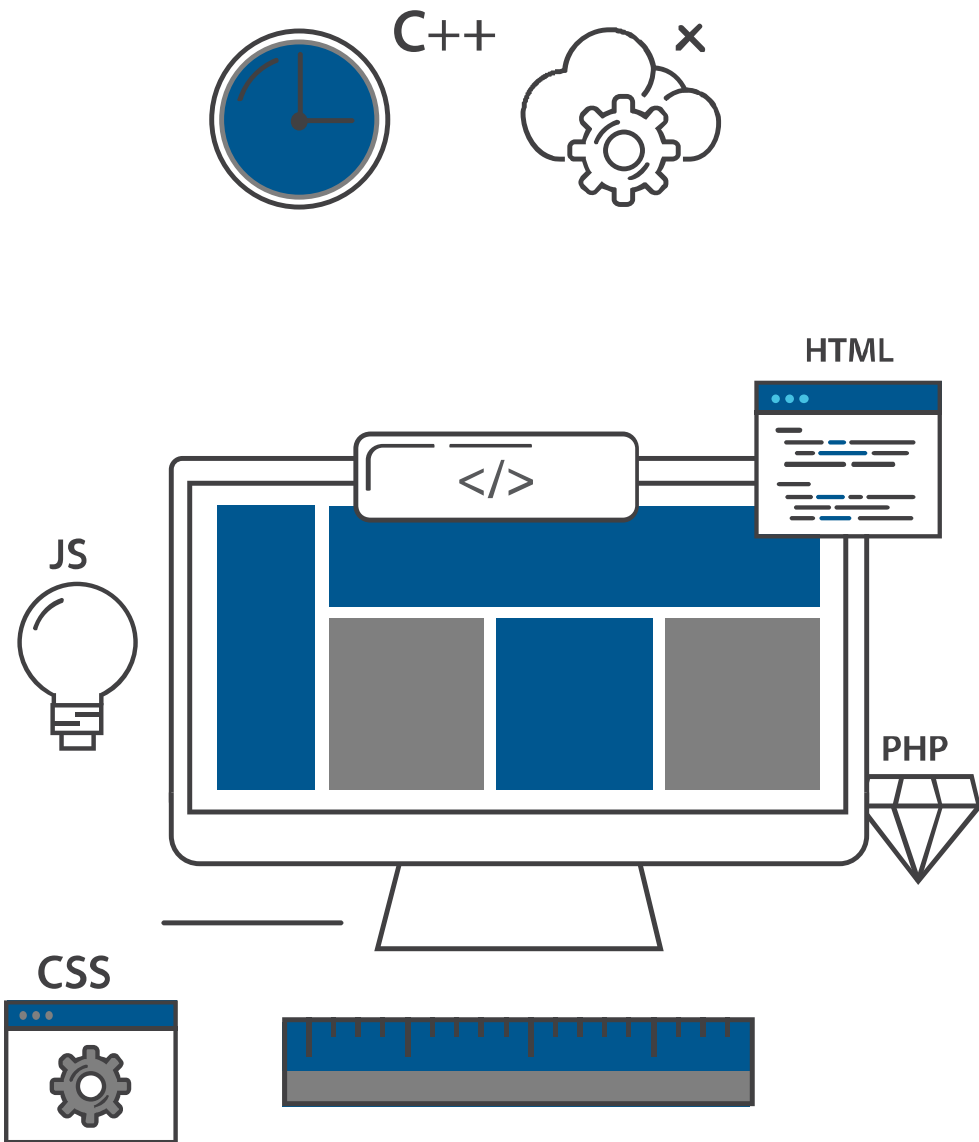


第三章 图

李翔

<https://implus.github.io/>

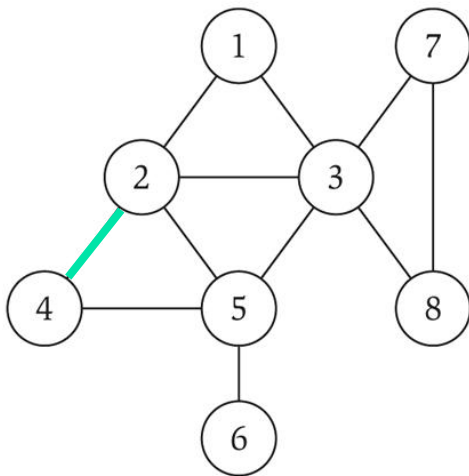




3.1 基本定义与应用

- 无向图 $G = (V, E)$
 - V = 顶点
 - E = 边, 反映顶点之间的关系
 - 图参数: $n = |V|, m = |E|$.

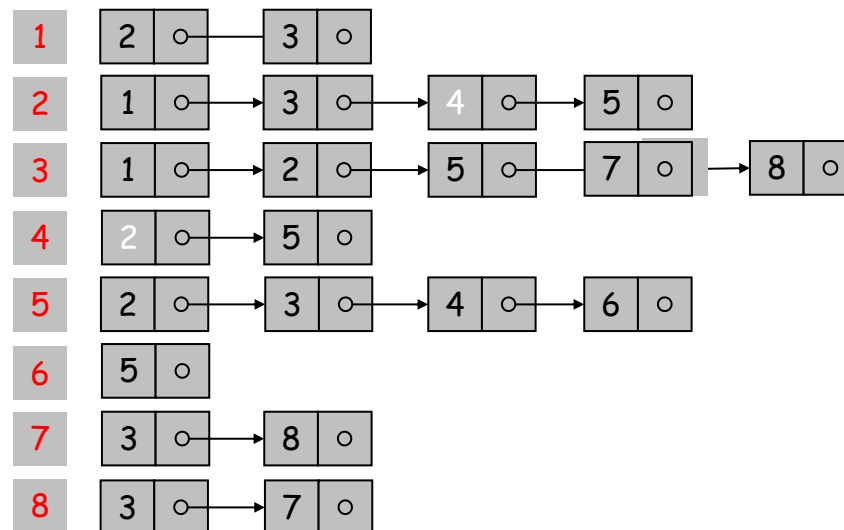
图的表示



邻接矩阵

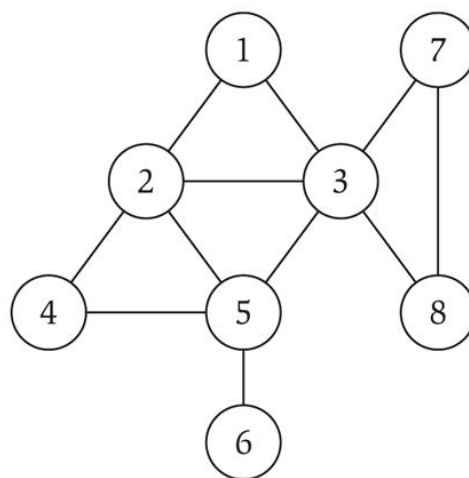
	1	2	3	4	5	6	7	8
1	0	1	1	0	0	0	0	0
2	1	0	1	1	1	0	0	0
3	1	1	0	0	1	0	1	1
4	0	1	0	1	1	0	0	0
5	0	1	1	1	0	1	0	0
6	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	1
8	0	0	1	0	0	0	1	0

邻接链表



圈的定义

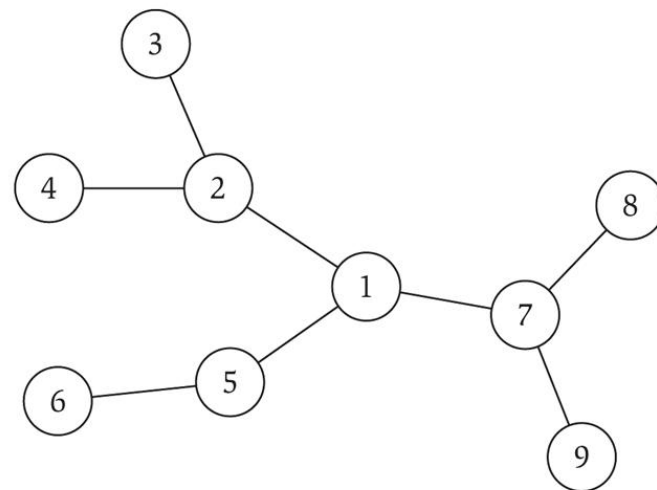
- Def. 路径 $v_1, v_2, \dots, v_{k-1}, v_k$ 被称为一个圈(Cycle), 如果 $v_1 = v_k$, $k > 2$, 而且前 $k-1$ 个顶点两两不同。



cycle $C = 1-2-4-5-3-1$

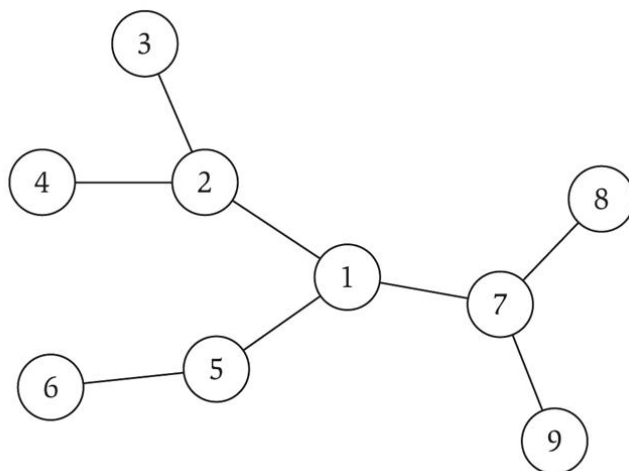
树的定义

- Def. 一个无向图被称为是树，如果它是连通的而且没有圈。
- 定理. 设 G 是一个具有 n 个结点的无向图。下面任意两条都可以推出第三条。
 - G 是连通的。
 - G 不包含一个圈。
 - G 有 $n-1$ 条边。

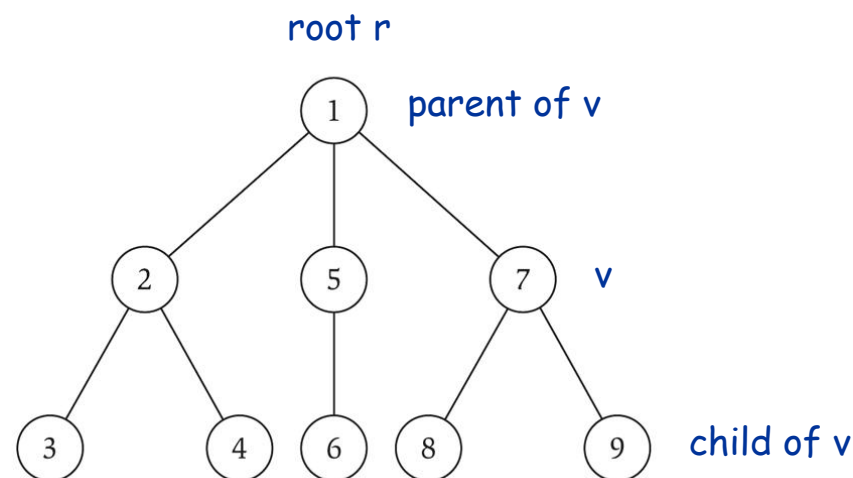


有根树

- 给一个树 T , 选择一个根节点 r , 安排所有的边从 r 出发.
- 目的是为了建立分层结构。



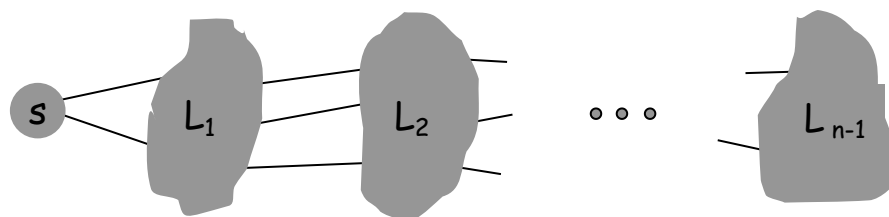
a tree



the same tree, rooted at 1

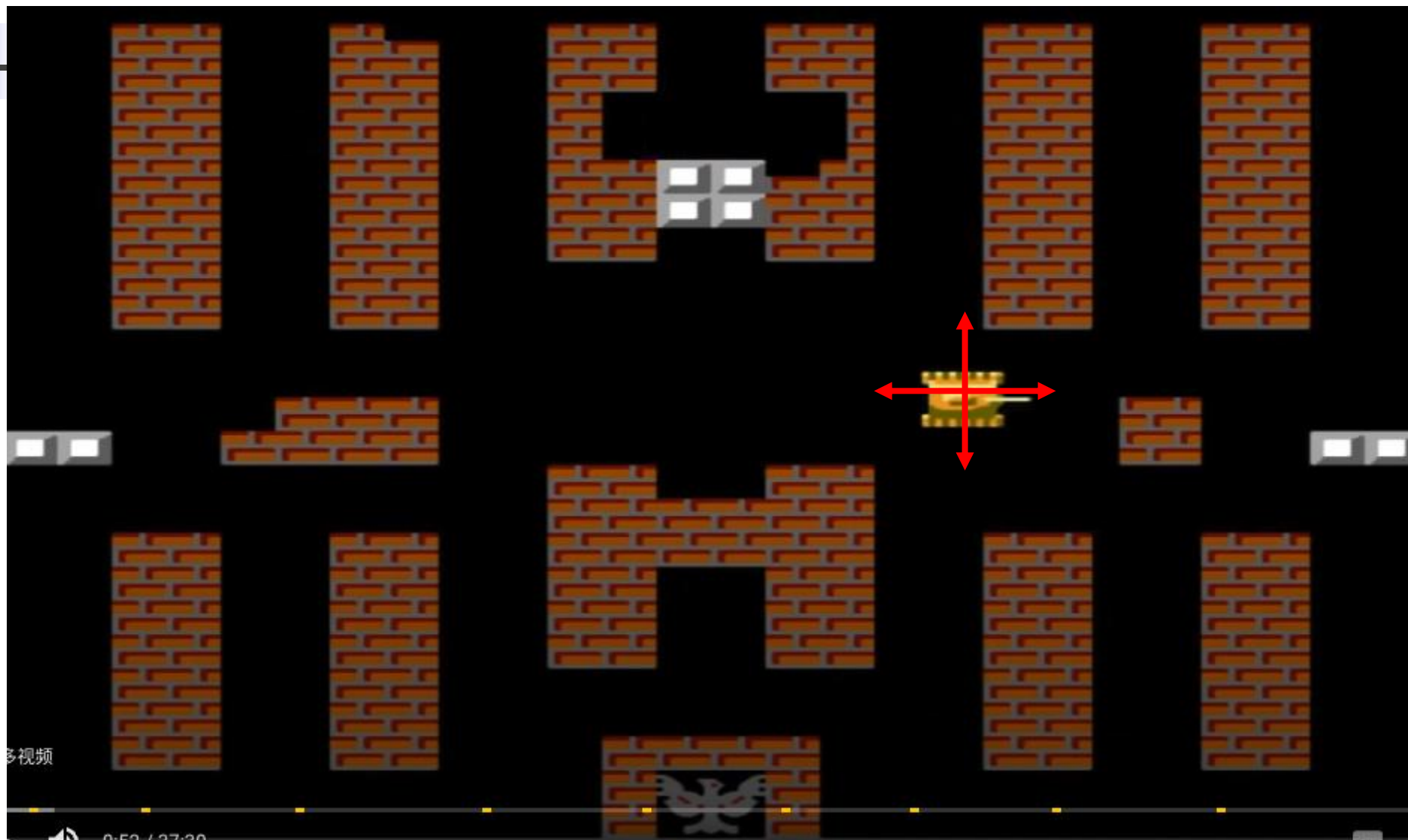
广度优先(Breadth-First Search)

- BFS算法简介. 从顶点 s 开始向各个方向“探索”，把探索到的顶点依次加在各层上。



- BFS algorithm.
 - $L_0 = \{ s \}$.
 - L_1 = all neighbors of L_0 .
 - L_2 = all nodes that do not belong to L_0 or L_1 , and that have an edge to a node in L_1 .
 - L_{i+1} = all nodes that do not belong to an earlier layer, and that have an edge to a node in L_i .

一个典型的BFS应用场景



ZTE中兴图灵之坦克大战



选择左边AI

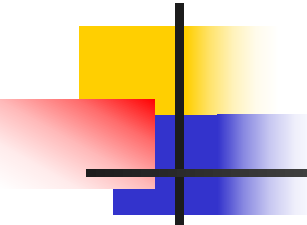
KeyBoard
AIDebug.dll
aurora.dll
K-2.dll
suixin.dll
TTMQLZ.dll
TTMQLZ1.dll
Yumir.dll

VS

选择右边AI

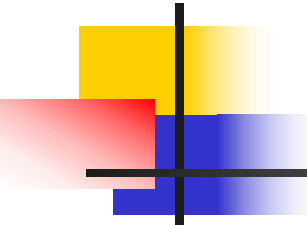
KeyBoard
AIDebug.dll
aurora.dll
K-2.dll
suixin.dll
TTMQLZ.dll
TTMQLZ1.dll
Yumir.dll

Start WebSite Exit



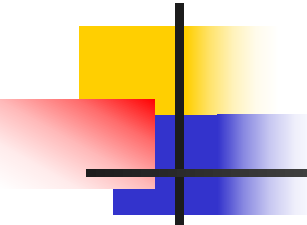
BFS性质

- BFS(s):
- 置Discovered[s]=true, 其他v, 置Discovered[v]=false
- 初始化L[0], 单个元素s构成
- 置层计数器i=0
- 置BFS树T=空集
- While L[i]非空
- 初始化一个空表L[i+1]
- For 每个结点L[i]中结点u
- 考虑每条关联到u的边(u,v)
- If Discovered[v]=false then
- 置 Discovered[v]=true
- 把边(u,v)加到树T上
- 把v加到表L[i+1]
- Endif
- Endfor
- 层计数器加1
- Endwhile



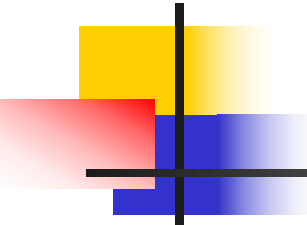
BFS性质

- 定理3. 11 如果图是邻接表给出, **BFS**算法的上述实现将以 $O(m+n)$ 时间运行。



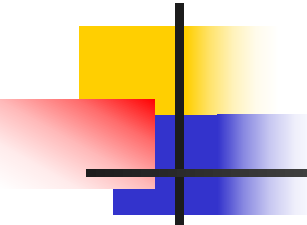
BFS 性质

- 定理 BFS算法产生的层 L_i 就是到源点 s 距离为 i 的顶点集合. 存在一条 s 到 t 的路径当且仅当 t 出现在某一层中.
- 定理 设 T 是图 $G = (V, E)$ 的一棵宽度优先搜索树, (x, y) 是 G 中的一条边.那么 x , y 所属的层数至多相差 1.



BFS & DFS

- BFS: 从s开始出发探查, 按照距离分层;
- DFS: 从s开始出发探查, 方式更“冲动”;
- BFS: 一般用**队列**来维护即将被处理的结点;
- DFS: 用**栈**来维护即将被处理的结点;

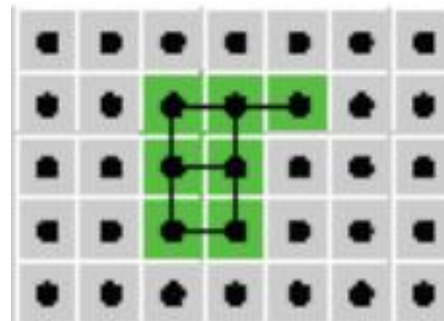


DFS

- DFS(s)
- 初始化S为具有一个元素s的栈
- While S 非空
 - 从S中取出一个节点u
 - If Explored[u]=false then
 - 置Explored[u]=true
 - For每条与u关联的边(u,v)
 - 把v加到栈S
 - Endfor
- EndIf
- EndWhile

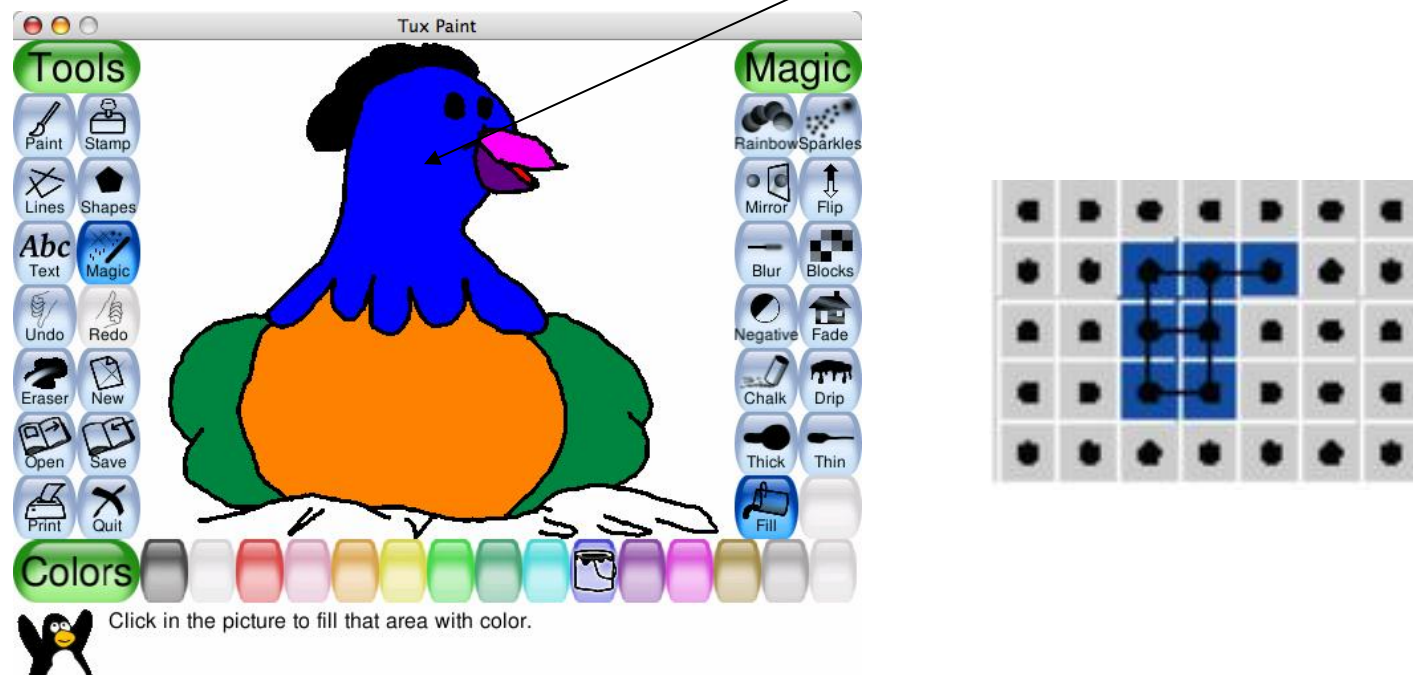
应用实例

- 图形填充问题：亮绿色→蓝色
- 顶点：像素点；边：是否都是亮绿色



应用实例

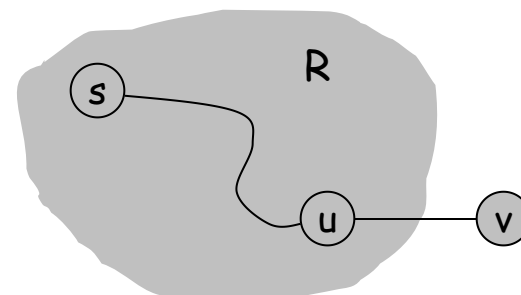
需要找到同种亮绿色的结构，然后改变颜色



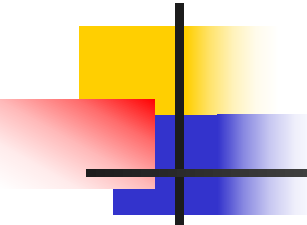
连通分支

- BFS算法发现的是从始点 s 可达的结点。
把这个集合 R 看作 G 的包含 s 的连通分支。

```
R will consist of nodes to which s has a path
Initially  $R = \{s\}$ 
While there is an edge  $(u, v)$  where  $u \in R$  and  $v \notin R$ 
    Add  $v$  to  $R$ 
Endwhile
```



定理： 算法结束产生的集合 R 恰好是
 G 的包含 s 的连通分支。

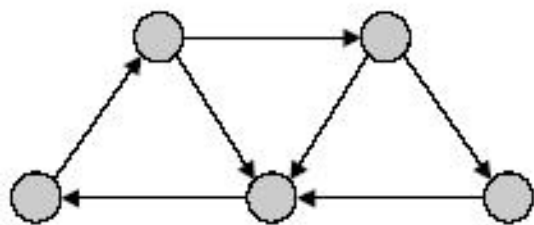


连通性质

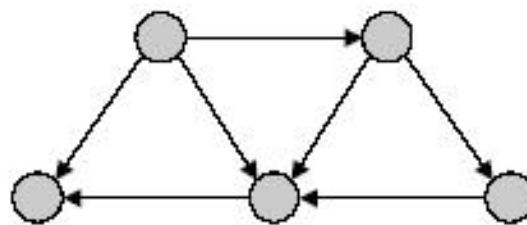
- 定理3.8: 对无向图中任两个结点 s 与 t , 它们的连通分支或者相等, 或者不相交。
- 定理3.17 对有向图中的任何两个结点 s 与 t , 它们的**强连通**分支或者相等, 或者不相交。

强连通性

- Def. u 和 v 是相互可达的，如果彼此之间存在到达对方的路径。
- Def. 如果图中每对结点是相互可达的，那么此图是强连通的。



strongly connected



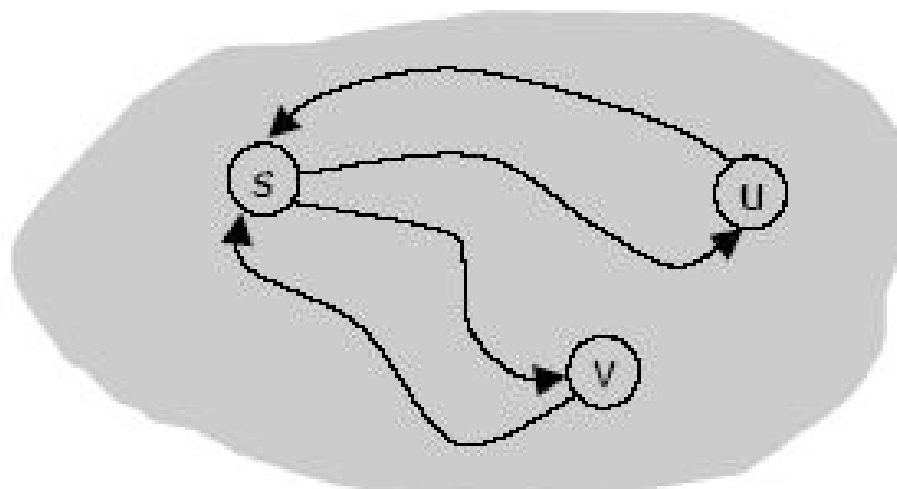
not strongly connected

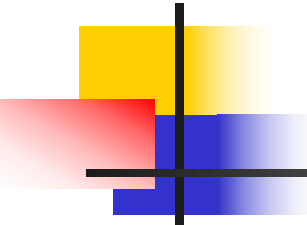
如何判断一个图是否强连通？所需时间代价？

强连通性

- 引理. 设 s 是图 G 中的任意一个结点。 G 是强连通的, 当且仅当图中的每个结点能够与 s 相互可达。

Pf.





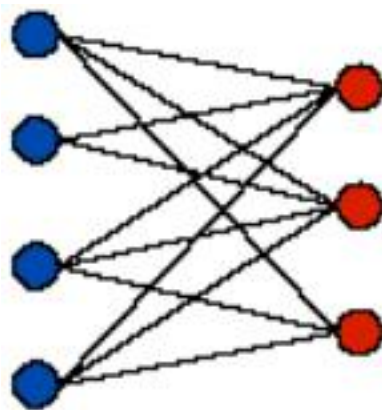
强连通性

- Pick any node s .
- Run BFS from s in G .
- Run BFS from s in G^{rev} .
- Return true iff all nodes reached in both BFS executions.

存在 $O(m + n)$ 的有效算法判别图 G 是否强连通。

3.4 二分性测试

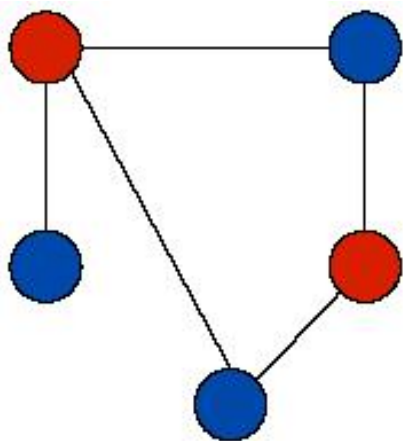
- 二部图是一个图，其中结点集可以划分为X与Y,每条边一端在X中，另一端在Y中。
- 直观的，一个图是二部图，如果能对结点着红色和蓝色，使得每条边有一个红端点和一个蓝端点。



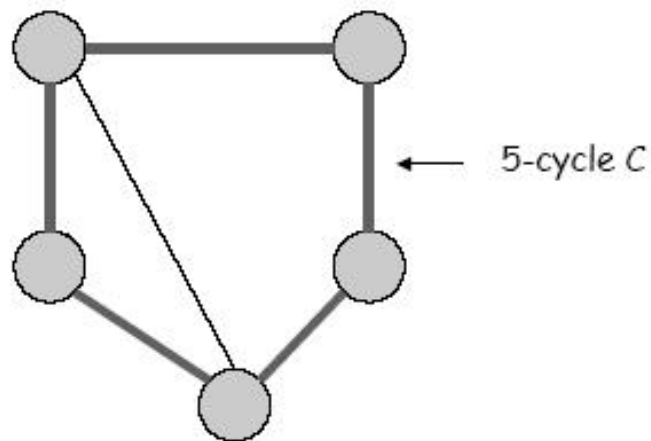
a bipartite graph

二部图性质

- 引理 图 G 是二部图当且仅当图中没有奇圈。



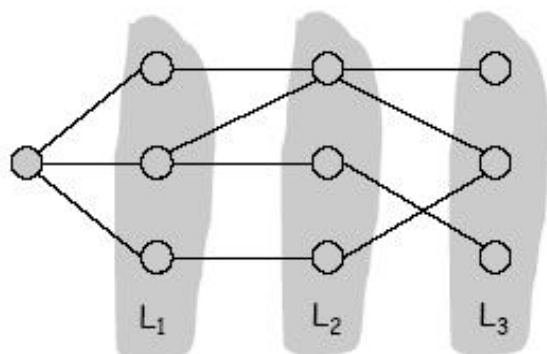
bipartite
(2-colorable)



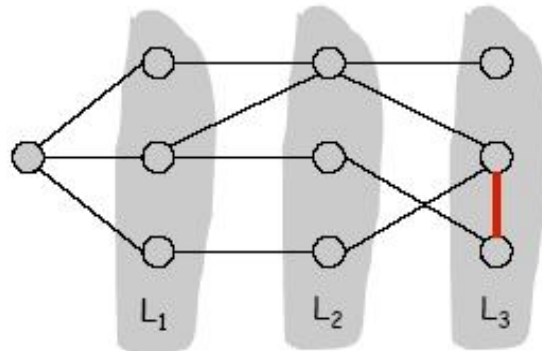
not bipartite
(not 2-colorable)

二部图性质

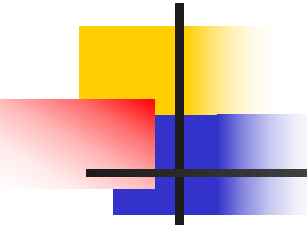
- 定理 设 G 是一个连通图， $L_0, \dots, L_k$ 是从顶点 s 由BFS算法生成的层。那么下面两件事一定恰好成立其一：
 - i. G 中没有边与同一层的两个结点相交。这种情况下 G 是二部图，其中偶数层的结点可以着红色，奇数层结点可以着蓝色。
 - ii. G 中有一条边与同一层的两个结点相交。此种情形下，存在一个奇圈，不可能是二部图。



Case (i)



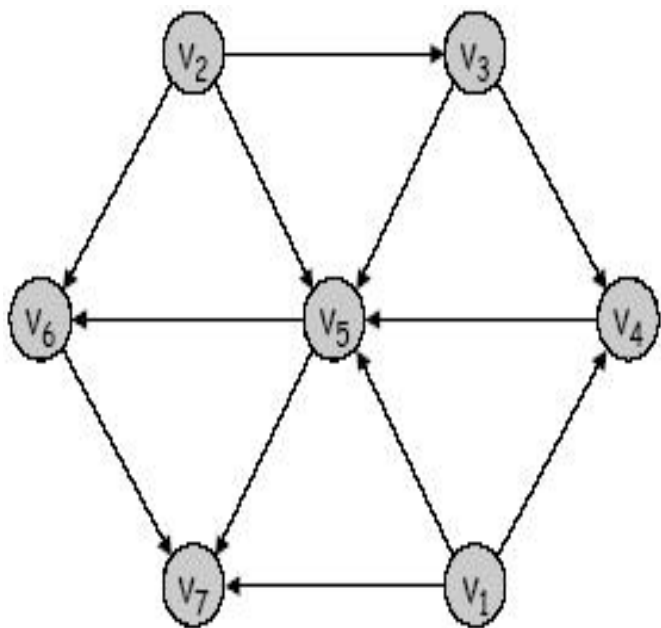
Case (ii)



3.6 有向无圈图与拓扑排序

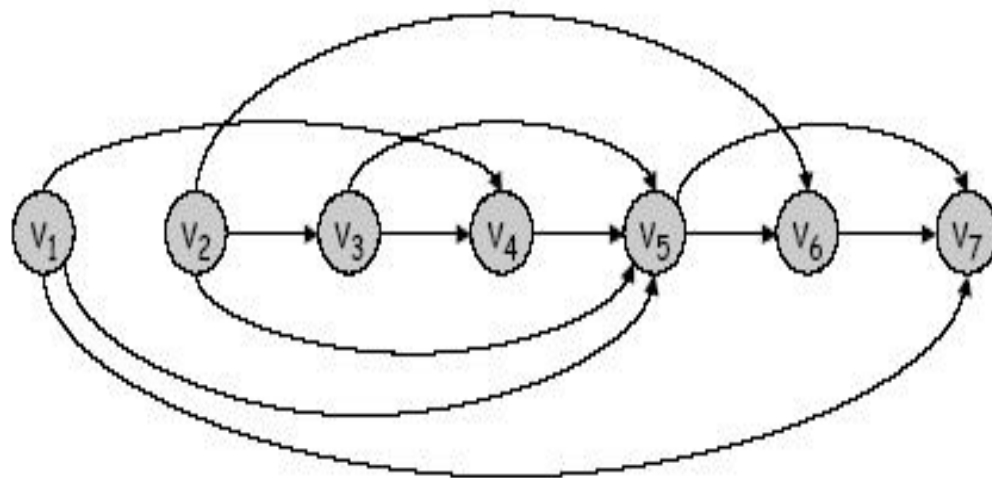
- 问题的背景：存在标记为 $\{1, 2, \dots, n\}$ 任务集，之间存在先后的依赖性，是否存在一个有效的执行任务的次序？
- 有向图 $G = (V, E)$ 的一个**拓扑排序**是它的结点作为 $v_1, v_2, \dots, v_n$ 的一个排序，对每条边 (v_i, v_j) ，都有 $i < j$.

拓扑排序



a DAG

等价!

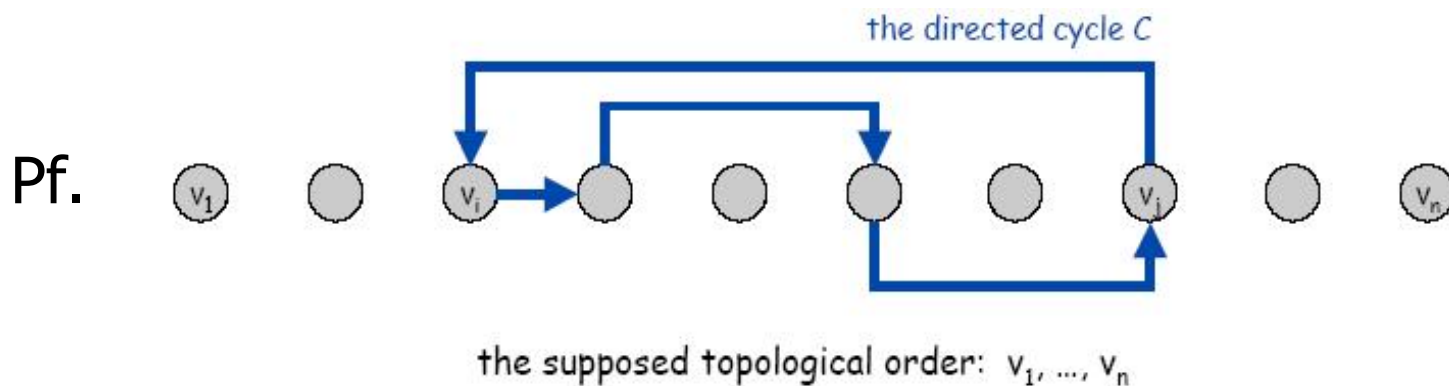


a topological ordering

有向无圈图与拓扑排序

- Def. **DAG**(Directed Acyclic Graphs):有向图没有圈

定理: 如果G有一个拓扑排序, 那么G是一个DAG.

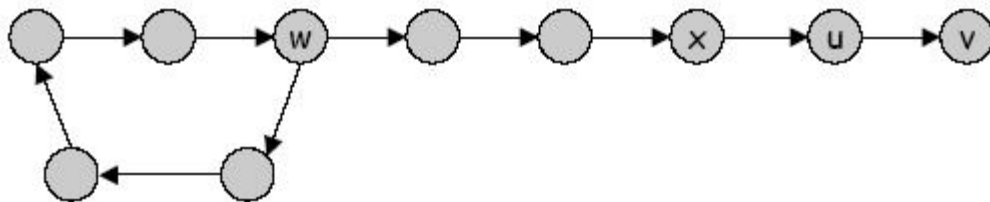


DAG

第一个结点

- 命题： 在每一个DAG中， 存在一个没有输入边的结点。

Pf.



定理 如果G是一个DAG, 那么G有一个拓扑排序。

Pf. 归纳证明 (利用命题)

方法：反证法

1. 假设相反：假设DAG中所有结点的入度均至少为1（即每个结点都有至少一条输入边）。

2. 构造无限路径：

- 任选一个结点 v_0 ，由于 v_0 的入度 ≥ 1 ，存在结点 v_1 使得 $v_1 \rightarrow v_0$ 。
- 同理， v_1 的入度 ≥ 1 ，存在结点 v_2 使得 $v_2 \rightarrow v_1$ 。
- 重复此过程，得到一条无限长的反向路径：

$$v_0 \leftarrow v_1 \leftarrow v_2 \leftarrow v_3 \leftarrow \cdots$$

3. 有限结点导致矛盾：

- 由于图中只有有限个结点（设为 n 个），在构造路径时，最多经过 $n + 1$ 步后，必然出现重复访问的结点（由鸽巢原理）。
- 设 $v_i = v_j$ ($i < j$)，则路径段 $v_i \leftarrow v_{i+1} \leftarrow \cdots \leftarrow v_j$ 形成一个环，即：

$$v_i \rightarrow v_{i+1} \rightarrow \cdots \rightarrow v_j \rightarrow v_i$$

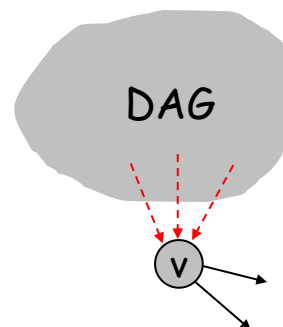
- 这与DAG的无环性矛盾。

4. 结论：原假设不成立，因此DAG中至少存在一个入度为0的结点。



■ 算法

To compute a topological ordering of G :
 Find a node v with no incoming edges and order it first
 Delete v from G
 Recursively compute a topological ordering of $G - \{v\}$
 and append this order after v



定理 上面算法在 $O(m + n)$ 时间内找到一个拓扑排序.

Pf. 考虑边逐次递减的代价 $O(m)$; 追踪被删除的结点代价 $O(n)$.